

تاسك: CRUD بالـ JavaScript من غير DOM

(1) الفكرة (Scenario)

أنت بتبني مدير منتجات لمحل **NTI Mini Store**. البيانات تعيش في الذاكرة فقط: `array of objects`. مفيش صفحة HTML بتضيف أو بتمسح المنتجات. كل الشغل من خلال `console` / `alert` / `confirm` / `prompt` + `functions`.

(2) قواعد إلزامية (Must Follow)

ممنوع في التاسك ده

- `document.getElementById`
- `innerHTML` / `innerText` لعرض المنتجات
- أزرار HTML تعمل `Create` / `Delete`
- مكتبات خارجية

مسموح

- `console.log` و `console.table`
- `prompt` / `confirm` / `alert`
- `Functions` بكل أنواعها اللي اتشرحت
- `Array of objects` + `array methods`
- `String methods` و `Number/Math methods`

(3) موديل البيانات (Data Model)

كل منتج يكون `object` بالشكل ده:

```
{
  id: 1,
  name: "Laptop",
  price: 18500.5,
```

```
category: "Electronics",
quantity: 4
}
```

خزن كل المنتجات في array اسمها products. الـ id يتولد تلقائي (auto increment) — المستخدم مبدخلوش بإيده.

4 المطلوب الأساسي (Required)

نفذ الـ functions التالية. كل function تعمل شغلها بـ return قدر الإمكان، والطباعة تكون في function لوحدها.

CRUD + Filter — الوظيفة المطلوبة

العملية	Function	المطلوب	اتطبق من الشرح
Create	createProduct(name, price, category, quantity)	تعمل trim للاسم والتصنيف. تتحقق إن الاسم مش فاضي، والسعر رقم أكبر من 0، والكمية عدد صحيح >= 0. لو البيانات غلط: ارجع رسالة خطأ (return) ومتضيفش حاجة. لو تمام: ضيف المنتج بـ push وارجع الـ object الجديد.	function declaration — default params — validation — string trim
Read all	getAllProducts()	ترجع الـ array كلها. اطبعها بـ console.table من function ثانية اسمها .printProducts(list).	return vs console.log — function calling another function
Read one	getProductById(id)	دور بـ .find. لو مش موجود ارجع null.	find — objects
Update	updateProduct(id, name, price, category, quantity)	لو القيمة فاضية أو المستخدم داس Cancel، سيب القيمة القديمة. عدّل نفس الـ object (mutable) وارجعه. لو الـ id مش موجود: رسالة واضحة.	findIndex — modifying object properties
Delete	deleteProduct(id)	اسأل بـ confirm قبل الحذف. لو وافق: امسح بـ splice وارجع المنتج المحذوف.	confirm — splice — findIndex

العملية	Function	المطلوب	اتطبق من الشرح
Filter / Search	filterProducts(keyword)	ابحث في name و category. لازم تستخدم نفس فكرة الديمو: الـ trim().toLowerCase().includes(...) array الأصلية متغيرش. ارجع array جديدة.	filter — toLowerCase — includes — trim — immutable

(5) قائمة تفاعلية (Interactive Menu)

اعمل function اسمها startApp() فيها حلقة while + switch. المستخدم يختار رقم من prompt:

```
===== NTI Mini Store =====
1) Add product
2) Show all products
3) Show product by ID
4) Update product
5) Delete product
6) Search / Filter
0) Exit
```

- الاختيار 0 أو Cancel → تخرج من الحلقة.
- أي رقم غلط → alert أو console.log برسالة واضحة، والحلقة تكمل.
- القائمة دي تستدعي الـ functions اللي فوق — مكتتبش لوجيك CRUD جوه الـ switch نفسه.

خلي startApp() متعملش run لوحدها في أول الملف وأنت بتجرب. اناديلها من الـ Console لما تبقى جاهز: startApp() عشان تقدر تختبر كل function لوحدها الأول.

(6) بيانات تجريبية (Seed Data)

ابدأ الـ array بـ 4 منتجات على الأقل عشان الـ filter يتختبر من غير ما تضيف بإيدك كل مرة:

```
var products = [
  { id: 1, name: "Laptop", price: 18500, category: "Electronics", quantity: 4 },
  { id: 2, name: "Mouse", price: 180, category: "Electronics", quantity: 25 },
  { id: 3, name: "Notebook", price: 35, category: "Stationery", quantity: 100 },
  { id: 4, name: "Desk Lamp", price: 420, category: "Home", quantity: 0 }
];
```

#	title	المطلوب	المفهوم
B1	منع الاسم المكرر	products.some(...) قبل الإضافة. المقارنة بعد trim().toLowerCase()	some
B2	ترتيب بالسعر	sortByPrice(order) — "asc" أو "desc" (default = asc). استخدم toSorted بشأن الأصلية متغيرش.	toSorted — default params
B3	إحصائيات المحل	getStoreStats() ترجع object فيه: عدد المنتجات، مجموع قيمة المخزون price * quantity بـ reduce، متوسط السعر بـ toFixed(2)، وعدد المنتجات الـ out of stock.	reduce — toFixed — object return
B4	تجميع حسب التصنيف	groupByCategory() بنفس فكرة reduce بتاعة users/city في الديمو.	reduce grouping
B5	Filter بالسعر	filterByPriceRange(min, max) — الكمية اللي من غير حد تبقى مفتوحة (default).	filter — default params
B6	Arrow + HOF	اعمل filterOdds style: const inStock = (list) => list.filter(p => p.quantity > 0) و function withAfterAction(fn, callback) تنادي callback بعد العملية (مثلاً تطبع "Done").	arrow — callback — HOF
B7	IIFE + Methods	لف التطبيق في IIFE، وخلي العمليات methods جوه object اسمه store.	IIFE — object methods
B8	Rest params	addMany(...items) تضيف أكثر من منتج مرة واحدة.	rest parameters

8) مثال تشغيل متوقع في الـ Console

```
createProduct("Keyboard", 350, "Electronics", 12)
// → { id: 5, name: "Keyboard", price: 350, category: "Electronics", quantity: 12 }

createProduct(" ", 10, "Home", 1)
// → "Error: name is required"
```

```
filterProducts(" elec ")
// → Laptop, Mouse, Keyboard (original products array unchanged)

getProductById(4)
// → Desk Lamp (quantity: 0)

deleteProduct(99)
// → "Error: product not found"

startApp()
// → menu with prompt, until user chooses 0
```

(9) نصائح

- متخلّطش بين `console.log` و `return` الـ CRUD functions ترجع قيمة. الطباعة في `printProducts`.
- الـ `filter` لازم يرجع **array** جديدة. لو استخدمت `splice` جوه `filter` هتكسر الداتا.
- `prompt` بيرجع `string` أو `null`. حوّل السعر بـ `Number` أو `parseFloat` وتحقق بـ `isNaN`.
- لو اتكرّرت في البحث، راجع مثال الفاكهة في `arrayObj.js` — نفس الفكرة بالظبط.
- لو اتكرّرت في شكل الـ `function`، راجع جدول `functionsTable.html`.

NTI JavaScript Track — Task: CRUD Operations by JS without DOM